

5. La ricorsione in coda

- ☐ Non è implementabile nei linguaggi imperativi
- ☐ **Nessuna delle altre risposte**
- ☐ Richiede di non scrivere mai la chiamata ricorsiva come ultimo statement di una subroutine
- ☐ Richiede di non ritornare mai direttamente il valore ritornato da una chiamata ricorsiva
- ☐ Permette di risolvere il problema della ricorsione infinita

6. Call by constant

- ☐ Fa una copia della variabile
- ☐ **è utile per passare oggetti**
- ☐ Ritorna il valore al chiamante
- ☐ Permette la modifica della variabile
- ☐ Nessuna delle altre risposte

9. I record di attivazione

- ☐ Sono necessari solo in presenza di funzioni di ordine superiore
- ☐ Sono allocati dinamicamente solo in caso di scope dinamico
- ☐ **Nessuna delle altre risposte**
- ☐ Devono essere esplicitamente allocati e deallocati dal codice del programma che li usa
- ☐ Sono allocati solo nello heap

10. Si può dire che una macchina astratta che capisce il linguaggio C non sia implementata in modo puramente compilativo perché

- ☐ Gli eseguibili generati da un compilatore C in genere non eseguono direttamente sulla macchina hardware, ma su una macchina astratta che include il runtime del linguaggio e le funzionalità del Sistema Operativo
- ☐ Gli eseguibili generati dal compilatore vengono comunque interpretati da una macchina virtuale
- ☐ Una macchina astratta che capisca un linguaggio di alto livello come il C non è mai implementabile con un compilatore
- ☐ Il runtime del linguaggio C è comunque sempre interpretato

☐ **Nessuna delle altre risposte**

11. Il costrutto for dei linguaggi C, C++ e Java

- ☐ **Non è un costrutto di iterazione determinata**
- ☐ E' necessario a tali linguaggi per implementare qualsiasi tipo di algoritmo
- ☐ Nessuna delle altre risposte
- ☐ E' un costrutto di iterazione determinata

- ☐ Permette di sapere in anticipo quante volte il ciclo verrà ripetuto (indipendentemente dal corpo del ciclo)

12. Un'entità esprimibile è

- ☐ Nessuna delle altre risposte
- ☒ **Un'entità che può essere generata come risultato da un'espressione complessa o da una funzione**
- ☐ Un'entità che ancora non compare nell'ambiente
- ☐ Un'entità che può essere memorizzata
- ☐ Una generica entità a cui può essere dato un nome

13. In assenza di ambiente non locale

- ☐ Per implementare funzioni definite ricorsivamente è necessario utilizzare un fixed point combinator
- ☐ Non si possono implementare algoritmi ricorsivi
- ☒ **Nessuna delle altre risposte**
- ☐ Non si possono implementare algoritmi iterativi o ricorsivi
- ☐ Non si possono implementare algoritmi iterativi

14. I puntatori di catena dinamica contenuti in un record di attivazione

- ☐ Collegano una lista di zone di memoria gestita dinamicamente
- ☐ Servono per accedere alle variabili dinamiche
- ☐ Non esistono "puntatori di catena dinamica" in un record di attivazione
- ☐ Nessuna delle altre risposte
- ☒ **Permettono, a partire da un RdA, di trovare il RdA precedente sullo stack**

15. In presenza di variabili modificabili

- ☐ Nessuna delle altre risposte
- ☒ **Esistono un Ambiente che associa valori denotabili (fra cui le locazioni di memoria) a nomi ed una Memoria che associa locazioni di memoria a valori memorizzabili**
- ☐ Non esistono valori denotabili
- ☐ La valutazione del comando di assegnamento restituisce sempre un valore
- ☐ Il comando di assegnamento non ha effetti collaterali

17. Il concetto di variabile modificabile

- ☐ E' l'unico concetto utilizzabile quando si parla di variabili
- ☐ E' imposto dall'architettura di Von Neumann (variabili non modificabili richiederebbero macchine astratte caratterizzate da memoria a sola lettura)
- ☐ Nessuna delle altre risposte
- ☒ **E' tipico del paradigma di programmazione imperativo**

- ☐ Permette di evitare il fenomeno dell'aliasing

18. Nella sostituzione ($\lambda a.abc$)[arrg/c]

- ☐ **E' necessario applicare una Alfa-equivalenza per evitare una cattura di variabile**
- ☐ Viene catturata la variabile c
- ☐ Si rischia di catturare la variabile "a" ed è necessario applicare Beta-equivalenza per risolvere in problema
- ☐ Non c'è alcuna cattura di variabile
- ☐ Nessuna delle altre risposte

20. Il fenomeno della cattura di variabili

- ☐ Nessuna delle altre risposte
- ☐ Non può essere evitato in alcun modo
- ☐ **Fa sì che dopo una sostituzione una variabile libera diventi legata (per esempio da un'astrazione $\lambda x.$)**
- ☐ Comporta la "sparizione" di variabili libere durante un'astrazione funzione
- ☐ E' dovuto all'assenza di un ambiente non locale

21. Una Macchina Astratta ML (LO) è

- ☐ E' un modo per descrivere un interprete
- ☐ Nessuna delle altre risposte
- ☐ E' implementabile solo basandosi sull'architettura di Von Neumann
- ☐ E' un modo per descrivere un compilatore
- ☐ **E' associata ad un proprio linguaggio macchina L, che è in grado di capire ed eseguire**

23. Il passaggio di parametri per nome

- ☐ Permette di passare valori solo dal chiamante al chiamato (e non viceversa)
- ☐ Nessuna delle altre risposte
- ☐ **E' implementabile passando una chiusura come parametro**
- ☐ Ha un valore solo teorico e non è implementabile in pratica
- ☐ Permette la cattura di variabili libere in modo da effetti non deterministici

24. La tecnica del display

- ☐ Nessuna delle altre risposte
- ☐ Permette di implementare facilmente lo scope dinamico
- ☐ **Permette di ridurre il costo derivante dalla scansione della catena statica quando si implementa lo scope statico**
- ☐ Permette di visualizzare le zone di memoria allocata dinamicamente
- ☐ Permette di implementare le regole di scope statico senza generare frammentazione della memoria

26. Un compilatore da un linguaggio L ad un linguaggio LO è

☐ **Un programma che trasforma un programma PL (espresso nel linguaggio L) in un programma PLO (espresso nel linguaggio LO) tale che per ogni input I si ha $PL(I) = PLO(I)$**

☐ Nessuna delle altre risposte

☐ Un programma scritto nel linguaggio LO che riceve come ingresso un programma PL (espresso nel linguaggio L) ed il suo input I generando lo stesso output che genera PL con input I

☐ L'implementazione di una macchina astratta scritta nel linguaggio LO, che capisce programmi scritti nel linguaggio L

☐ Una implementazione di macchine astratte indipendente dalla macchina fisica

27. Il costrutto for dei linguaggi C, C++ e Java non è un costrutto di iterazione determinata perchè

☐ L'esistenza di costrutti di iterazione determinata implicherebbe che C, C++ e Java non sono Turing-completi

☐ Nessuna delle altre risposte

☐ **Dall'interno del ciclo è possibile modificare il valore del contatore**

☐ Non esistono costrutti di iterazione determinata

☐ C, C++ e Java sono linguaggi imperativi

28. Un oggetto denotabile (intendendo per "oggetto" una generica entità che può essere una variabile, una funzione, etc...) è

☐ Un "oggetto" che può essere memorizzato in una variabile

☐ Un "oggetto" per cui compare un binding nell'ambiente

☐ **Nessuna delle altre risposte**

☐ Un "oggetto" che ancora non compare nell'ambiente

☐ Un "oggetto" che può essere generato come risultato da un'espressione complessa o da una funzione

29. L'ambiente non locale di un blocco di codice è

☐ Nessuna delle altre risposte

☐ L'insieme dei valori che le variabili non locali possono assumere

☐ L'insieme dei binding creati all'interno del blocco di codice

☐ **L'insieme dei binding visibili dentro al blocco, ma non direttamente definiti in esso**

☐ Il subset dell'ambiente non visibile dentro al blocco di codice

32. I puntatori di catena dinamica contenuti in un record di attivazione

☐ **Nessuna delle altre risposte**

☐ Non esistono "puntatori di catena dinamica" in un record di attivazione

☐ Servono per identificare la zona di memoria in cui è memorizzata una variabile locale

- ☐ Devono essere esplicitamente allocati e deallocati dal codice del programma che li usa
- ☐ Servono per accedere alle variabili dinamiche

34. L'allocazione dinamica della memoria

- ☐ Nessuna delle altre risposte
- ☐ E' sempre effettuata solo dal compilatore o dall'interprete
- ☐ Può essere fatta solo dallo heap
- ☐ Può essere fatta solo dallo stack
- ☐ **Può essere fatta sia dallo stack che dallo heap**

35. Un compilatore da un linguaggio L ad un linguaggio LO è

- ☐ **Nessuna delle altre risposte**
- ☐ Una implementazione di macchine astratte indipendente dalla macchina fisica
- ☐ Un programma che trasforma un programma PLO (espresso nel linguaggio LO) in un programma PL (espresso nel linguaggio L) tale che per ogni input I si ha $PL(I) = PLO(I)$
- ☐ L'implementazione di una macchina astratta scritta nel linguaggio LO, che capisce programmi scritti nel linguaggio L
- ☐ Un programma scritto nel linguaggio LO che riceve come ingresso un programma PL (espresso nel linguaggio L) ed il suo input I generando lo stesso output che genera PL con input I

36. I dangling pointer

- ☐ Sono identificabili tramite tecniche di reference counting (contatore dei riferimenti)
- ☐ Sono un problema solo per il linguaggio Java
- ☐ **Sono identificabili tramite la tecnica detta "mark and sweep"**
- ☐ Non possono essere identificati con certezza, ma questo non è un problema perché comportano solo un piccolo spreco di memoria
- ☐ Nessuna delle altre risposte

37. La frammentazione esterna causa

- ☐ Uno spreco di memoria
- ☐ Nessuna delle altre risposte
- ☐ **L'impossibilità di allocare grandi blocchi di memoria anche se la memoria libera totale è sufficiente**

- ☐ Un rallentamento rilevante nelle operazioni di allocazione della memoria
- ☐ Il funzionamento non corretto di programmi che allocano memoria dinamicamente

38. La valutazione con corto circuito del predicato "A && B" (dove "&&" rappresenta un "AND" logico)

- ☐ Stabilisce che se "B" è vero allora "A" non viene valutato
- ☐ Nessuna delle altre risposte

- ☐ **Stabilisce che se "A" è falso allora "B" non viene valutato**
- ☐ Crea un non-determinismo nell'ordine della valutazione di "A" e "B"
- ☐ Stabilisce che "A" e "B" devono essere valutati in parallelo

39. Se l'ambiente di una funzione non contiene il nome della funzione stessa

- ☐ Nessuna delle altre risposte
- ☐ **Non è possibile per la funzione invocarsi ricorsivamente**
- ☐ Non ci sono particolari conseguenze
- ☐ La funzione non può usare scope dinamico
- ☐ La funzione non può usare scope statico

41. La frammentazione interna causa

- ☐ Il funzionamento non corretto di programmi che allocano memoria dinamicamente
- ☐ Nessuna delle altre risposte
- ☐ Un rallentamento rilevante nelle operazioni di allocazione della memoria
- ☐ **Uno spreco di memoria**
- ☐ L'impossibilità di allocare grandi blocchi di memoria anche se la memoria libera totale è sufficiente

42. L'allocazione dinamica della memoria

- ☐ **Può essere fatta sia dallo stack che dallo heap**
- ☐ Nessuna delle altre risposte
- ☐ E' sempre effettuata solo dal compilatore o dall'interprete
- ☐ Può essere fatta solo dallo stack
- ☐ Può essere fatta solo dallo heap

45. L'ambiente (o environment) è

☐ **L'insieme delle associazioni (nome, entità denotabile) esistenti in uno specifico punto del programma ed in uno specifico momento durante l'esecuzione di un programma**

- ☐ L'insieme dei valori che una variabile assume durante l'esecuzione di un programma
- ☐ Un insieme di associazioni (nome, valore) definite staticamente durante lo sviluppo di un programma
- ☐ Nessuna delle altre risposte
- ☐ Una lista di coppie (nome, tipo) che permette di accedere alle variabili di un programma

47. Si può dire che una macchina astratta che capisce il linguaggio Java non sia implementata in modo puramente compilativo perché

- ☐ **Non esistendo un vero e proprio runtime per Java, non si può parlare di compilazione pura**

- ☐ La macchina virtuale di Java (JVM) deve comunque essere compilata
- ☐ Una macchina astratta che capisca un linguaggio di alto livello come Java non è mai implementabile con un compilatore
- ☐ Non esistono compilatori Java
- ☐ Nessuna delle altre risposte

48. La ricorsione in coda

- ☐ Permette di risolvere il problema della ricorsione infinita
- ☐ Richiede di non scrivere mai la chiamata ricorsiva come ultimo statement di una subroutine
- ☐ Richiede di non ritornare mai direttamente il valore ritornato da una chiamata ricorsiva
- ☐ Nessuna delle altre risposte
- ☐ **Permette di evitare un'eccessiva crescita della dimensione dello stack**

49. La memoria gestita staticamente

- ☐ E' allocata esplicitamente dal programma a tempo di esecuzione, ma una volta allocata è staticamente legata al programma e non può essere liberata fino alla sua terminazione
- ☐ E' allocata prima dell'esecuzione del programma. Le entità allocate staticamente possono essere deallocate durante l'esecuzione del programma, per liberare memoria
- ☐ **E' allocata dal compilatore prima dell'esecuzione del programma. Le entità allocate staticamente in memoria risiedono in una zona fissa di memoria durante tutta l'esecuzione del programma**
- ☐ E' una memoria a sola lettura
- ☐ Nessuna delle altre risposte

52. Un interprete di un linguaggio L scritto in un linguaggio LO è

- ☐ Nessuna delle altre risposte
- ☐ L'implementazione di una macchina astratta scritta nel linguaggio LO, che capisce programmi scritti nel linguaggio L
- ☐ Una implementazione di macchine astratte indipendente dalla macchina fisica
- ☐ **Un programma scritto nel linguaggio LO che riceve come ingresso un programma PL (espresso nel linguaggio L) ed il suo input I generando lo stesso output che genera PL con input I**
- ☐ Un programma che trasforma un programma PL (espresso nel linguaggio L) in un programma PLO (espresso nel linguaggio LO) tale che per ogni input I si ha $PL(I) = PLO(I)$

53. In caso di scope statico

- ☐ Non è possibile annidare più blocchi di istruzioni
- ☐ I legami fra nomi ed oggetto possono essere determinati solo a tempo di esecuzione
- ☐ Il valore assegnato ad una variabile non può essere modificato

☐ **I legami fra nomi ed oggetto possono essere determinati semplicemente leggendo il testo di un programma**

☐ Nessuna delle altre risposte

54. Un garbage collector

☐ **Può essere implementato tramite la tecnica detta "mark and sweep", che riesce sempre ad identificare tutta la memoria allocata dinamicamente ma non più utilizzata**

☐ Richiede un'implementazione complessa, usando la tecnica dei tombstone (pietre tombali)

☐ E' implementabile solo in linguaggi di programmazione funzionali

☐ E' implementabile tramite la tecnica di lucchetti e chiavi, che però può causare dei memory leak

☐ Nessuna delle altre risposte

57. Un interprete di un linguaggio L scritto in un linguaggio LO è

☐ Un programma che trasforma un programma PL (espresso nel linguaggio L) in un programma PLO (espresso nel linguaggio LO) tale che per ogni input I si ha $PL(I) = PLO(I)$

☐ **Nessuna delle altre risposte**

☐ Una implementazione di macchine astratte indipendente dalla macchina fisica

☐ Un programma scritto nel linguaggio L che dato un input I produce lo stesso output generato dallo stesso programma scritto nel linguaggio LO

☐ L'implementazione di una macchina astratta scritta nel linguaggio LO, che capisce programmi scritti nel linguaggio L

58. L'ambiente (o environment) è

☐ **Nessuna delle altre risposte**

☐ Un insieme di associazioni (nome, valore) definite staticamente durante lo sviluppo di un programma

☐ L'insieme delle associazioni (variabile, valore) esistenti in uno specifico punto del programma ed in uno specifico momento durante l'esecuzione di un programma

☐ L'insieme dei valori che una variabile assume durante l'esecuzione di un programma

☐ Una lista di coppie (nome, tipo) che permette di accedere alle variabili di un programma

61. Un garbage collector

☐ Richiede un'implementazione complessa, usando la tecnica dei tombstone (pietre tombali)

☐ E' implementabile solo in linguaggi di programmazione funzionali

☐ Può essere implementato tramite la tecnica del reference counting (contatore dei riferimenti), che riesce sempre ad identificare tutta la memoria allocata dinamicamente ma non più utilizzata

☐ E' implementabile tramite la tecnica di lucchetti e chiavi, che però può causare dei memory leak

☐ **Nessuna delle altre risposte**

65. L'ambiente (o environment) è

☐ Un insieme di associazioni (nome, valore) definite staticamente durante lo sviluppo di un programma

☐ Una lista di coppie (nome, tipo) che permette di accedere alle variabili di un programma

☐ Nessuna delle altre risposte

☐ L'insieme dei valori che una variabile assume durante l'esecuzione di un programma

☐ **L'insieme delle associazioni (nome, oggetto denotabile) esistenti in uno specifico punto del programma ed in uno specifico momento durante l'esecuzione di un programma**

66. In caso di scope dinamico

☐ Il valore assegnato ad una variabile non può essere modificato

☐ Non è possibile annidare più blocchi di istruzioni

☐ Nessuna delle altre risposte

☐ **I legami fra nomi ed oggetto possono essere determinati solo a tempo di esecuzione**

☐ I legami fra nomi ed oggetto possono essere determinati semplicemente leggendo il testo di un programma

68. Un entità denotabile è

☐ Nessuna delle altre risposte

☐ Un'entità che può essere generata come risultato da un'espressione complessa o da una Funzione

☐ **Una generica entità a cui può essere dato un nome**

☐ Un'entità che ancora non compare nell'ambiente

☐ Un'entità che può essere memorizzata

69. I puntatori di catena statica contenuti in un record di attivazione

☐ Nessuna delle altre risposte

☐ Servono per accedere alle variabili statiche

☐ Devono essere esplicitamente allocati e deallocati dal codice del programma che li usa

☐ **Servono per identificare la zona di memoria in cui è memorizzata una variabile in caso di scope statico**

☐ Non esistono puntatori di catena statica in un record d'attivazione

70. La memoria gestita dinamicamente

☐ E' usata solo in linguaggi interpretati

- ☐ Non è mai strettamente necessaria, ma permette di ottenere migliori prestazioni
- ☐ **E' necessaria per implementare la ricorsione**
- ☐ Nessuna delle altre risposte
- ☐ E' necessaria per implementare l'iterazione

75. Un entità denotabile può avere un tempo di vita più lungo di quello delle associazioni (fra l'entità ed identificatori) che lo riferiscono

- ☐ Nessuna delle altre risposte
- ☐ **Se l'entità è allocata dinamicamente dallo heap in una subroutine**
- ☐ Mai
- ☐ Se l'entità è allocata dinamicamente dallo stack in una subroutine
- ☐ Se si usa scope dinamico

76. La memoria gestita dinamicamente

- ☐ Non è mai strettamente necessaria, ma permette di ottenere migliori prestazioni
- ☐ E' necessaria per implementare l'iterazione
- ☐ E' usata solo in linguaggi interpretati
- ☐ E' necessaria solo per implementare macchine astratte per linguaggi compilati
- ☐ **Nessuna delle altre risposte**

77. I record d'attivazione

- ☐ Devono essere esplicitamente allocati e deallocati dal codice del programma che li usa
- ☐ **Possono essere allocati sia sullo stack sia sullo heap (in caso, per esempio, di funzioni di ordine superiore)**
- ☐ Sono necessari solo in presenza di funzioni di ordine superiore
- ☐ Solo allocati solo sullo heap
- ☐ Nessuna delle altre risposte

114. Quale delle seguenti affermazioni riguardanti il garbage collector è vera

- ☐ Per Garbage Collection si intende una modalità automatica di gestione della memoria
- ☐ L'utente alloca liberamente memoria
- ☐ Non è permesso deallocare memoria
- ☐ Il sistema periodicamente recupera la memoria allocata e non più utilizzabile
- ☐ Un sistema operativo, o un compilatore, e un modulo di run-time liberano porzioni di memoria non più utilizzate dalle applicazioni.
- ☐ **Tutte sono corrette**

115. Quale delle seguenti affermazioni riguardanti il Garbage Collector è falsa

- ☐ Il garbage collector annoterà le aree di memoria non più referenziate e le libererà automaticamente
- ☐ L'utente alloca liberamente memoria

- **E' permesso deallocare memoria**

- Un sistema operativo, o un compilatore, e un modulo di run-time liberano porzioni di memoria non più utilizzate dalle applicazioni.

- Il sistema periodicamente recupera la memoria allocata e non più utilizzabile